

Mini Exercise — Thực hành API Testing

Mục lục

- 1. Mục tiêu học tập
- 2. Bối cảnh và phạm vi
- 3. Chuẩn bị trước giờ học
- 4. Bước 1 — Generate with AI
- 5. Bước 2 — Audit (human review)
- 6. Bước 3 — Extend
- 7. Bước 4 — Execute (Postman + Newman)
- 8. Bước 5 — CI/CD
- 9. Bước 6 — Postman features
- 10. Thành phần bài nộp
- 11. Tài liệu tham khảo

1. Mục tiêu học tập

Sau bài thực hành, sinh viên có thể:

1. Thực hành trọn pipeline kiểm thử API tự động: **Generate** → **Audit** → **Extend** → **Execute** → **CI/CD**.
2. Dùng AI để đề xuất test case cho một API, sau đó tự đánh giá và sửa kết quả AI.
3. Chuyển test case đã duyệt thành dữ liệu chạy lặp trong Postman.
4. Chạy collection bằng Newman và đọc kết quả assertion từ báo cáo JSON.
5. Quan sát CI/CD pipeline pass và fail trên GitHub Actions.

Mô tả bài thực hành: Bài thực hành Mini Exercise này được thiết kế như một quy trình kiểm thử API tự động (API Testing Pipeline). Mục tiêu chính là giúp sinh viên làm quen và thực hành nhanh trọn vẹn luồng kiểm thử từ khâu sử dụng AI để sinh kịch bản kiểm thử, kiểm duyệt chất lượng thủ công, tự bổ sung các ca kiểm thử nâng cao, chạy kiểm thử tự động với Postman/Newman, cho đến thiết lập tích hợp liên tục (CI/CD) trên GitHub Actions. Mỗi sinh viên cần tự chọn một API từ hệ thống eshop-sut để thực hành.

2. Bối cảnh và phạm vi

Mỗi sinh viên tự chọn 1 API duy nhất từ danh sách phân hạng dưới đây để thực hiện bài tập. Không được trùng API với bạn cùng nhóm.

Danh sách các API để chọn lựa:

#	Endpoint	Mô tả & Lưu ý
1	<code>GET /api/products</code>	Lấy danh sách sản phẩm. Có hỗ trợ query string <code>?search=keyword</code> .
2	<code>GET /api/products/:id</code>	Lấy chi tiết 1 sản phẩm theo ID.
3	<code>POST /api/products</code>	Thêm mới sản phẩm. Yêu cầu JSON body đầy đủ thông tin.
4	<code>PUT /api/products/:id</code>	Cập nhật sản phẩm theo ID.
5	<code>DELETE /api/products/:id</code>	Xóa sản phẩm theo ID.
6	<code>GET /api/categories</code>	Lấy danh sách danh mục sản phẩm.
7	<code>POST /api/categories</code>	Thêm mới danh mục. Yêu cầu Auth Token.
8	<code>POST /api/register</code>	Đăng ký tài khoản mới. Cần test trường hợp email trùng, thiếu field.
9	<code>POST /api/login</code>	Đăng nhập hệ thống. Cần test trường hợp tài khoản đúng/sai/thiếu thông tin.
10	<code>POST /api/forgot-password</code>	Yêu cầu đặt lại mật khẩu.
11	<code>POST /api/reset-password</code>	Đặt lại mật khẩu bằng token.
12	<code>GET /api/users/me</code>	Lấy thông tin cá nhân. Yêu cầu Auth Token.
13	<code>PUT /api/users/me</code>	Cập nhật thông tin cá nhân. Yêu cầu Auth Token.
14	<code>GET /api/cart</code>	Xem giỏ hàng hiện tại. Yêu cầu Auth Token.
15	<code>POST /api/cart</code>	Thêm sản phẩm vào giỏ hàng. Yêu cầu Auth Token và JSON body.
16	<code>POST /api/checkout</code>	Thanh toán giỏ hàng. Yêu cầu Auth Token. Tạo đơn hàng mới với trạng thái <code>pending</code> .
17	<code>GET /api/orders/my-orders</code>	Xem danh sách đơn hàng của mình. Yêu cầu Auth Token.
18	<code>GET /api/orders/:id</code>	Xem chi tiết đơn hàng theo ID.
19	<code>PUT /api/orders/:id/cancel</code>	Hủy đơn hàng. Yêu cầu Auth Token. Có ràng buộc trạng thái (state transition).
20	<code>POST /api/apply-coupon</code>	Áp dụng mã giảm giá. Cần test mã hợp lệ/hết hạn/không tồn tại/đã dùng hết lượt.
21	<code>PUT /api/admin/orders/:id/status</code>	Admin cập nhật trạng thái đơn hàng. Có state transition: <code>pending → confirmed → shipping → delivered</code> .

Hệ thống có luồng trạng thái đơn hàng (state transition):

```

pending → confirmed → shipping → delivered
  \         \
  canceled   canceled

```

Phạm vi thực hành: Thiết kế và chạy bộ kịch bản kiểm thử tự động (Data-driven) cho **1 API** đã chọn từ bảng trên.

3. Chuẩn bị trước giờ học

3.1. Sinh viên

- Node.js 18 hoặc 20 LTS, npm và Git.
- Postman Desktop.
- Newman đã cài và có thể gọi bằng lệnh `newman --version`.
- **Quy trình Git:**
 1. Sử dụng repository của nhóm bạn đã **fork từ** `eshop-sut` của giảng viên.
 2. Truy cập tab **Actions** trên repository đó và đảm bảo đã bấm nút **"Enable GitHub Actions"**.
 3. **Clone** repository đó về máy cá nhân của mình và tạo một nhánh riêng ví dụ `feature/<MSSV>` để thực hành.
- Một công cụ AI có thể lưu lại prompt và output.

Từ thư mục gốc repository, kiểm tra nhanh:

```
node --version
```

```
npm --version
```

```
newman --version
```

If chưa cài dependencies, thực hiện trước buổi học:

```
cd backend  
npm install  
npm install --global newman
```

3.2. Các tệp cần tự thiết kế và chuẩn bị

Sinh viên sẽ tự thiết kế và tạo các tệp sau cho API đã chọn:

Tên tệp	Mô tả	Ghi chú
<code><ten-api>.postman_collection.json</code>	Postman Collection	Chứa kịch bản kiểm thử của API đã chọn (bao gồm các test assertions).
<code>local.postman_environment.json</code>	Cấu hình môi trường	Cấu hình biến <code>baseUrl</code> (mặc định <code>http://localhost:3000</code>) và <code>studentId = <MSSV></code> .
<code>mini-<api-name>.data.json</code>	Tập dữ liệu kiểm thử (Data-driven)	Chứa ít nhất 5 test case. Ví dụ: <code>mini-login.data.json</code> , <code>mini-cart.data.json</code> .
<code>newman-api-test.yml</code>	Tập cấu hình GitHub Actions (CI Workflow)	Dùng để kích hoạt chạy kiểm thử tự động bằng Newman khi push code.

4. Bước 1 — Generate with AI

Thực hiện Bước 1 "Generate with AI" với mục tiêu thiết kế ≥ 12 test case.

Mô tả API bạn đã chọn (endpoint, method, request/response mẫu, các trạng thái trả về) và gửi cho AI yêu cầu đề xuất ≥ 12 test case bao phủ:

- **Domain partitions:** các giá trị hợp lệ, không hợp lệ, biên (boundary) của từng tham số đầu vào (ví dụ: email format, password complexity, price > 0 , ID tồn tại/không tồn tại).
- **State transitions** (nếu API liên quan đến đơn hàng): kiểm tra luồng trạng thái `pending → confirmed → shipping → delivered`, các quy tắc hủy đơn, và các chuyển trạng thái không hợp lệ.
- **Security:** thiếu token, token hết hạn, token sai format, SQL injection, IDOR (truy cập tài nguyên của người khác), role escalation (nếu API yêu cầu xác thực).
- **Schema validation:** response body phải chứa đúng các field mà API trả về theo đặc tả.

Prompt phải yêu cầu AI trả về các cột: `tc_id`, input, expected status, expected fields và rationale. **Không dùng prompt kiểu "generate all tests"** — hướng dẫn AI thiết lập từng bước một cách chi tiết.

Tham khảo [postman-contract-test-prompt-guide.md](#) để biết cách viết prompt hiệu quả cho từng loại test case và tạo skill để dùng cho api testing.

5. Bước 2 — Audit (human review)

Thực hiện Bước 2 "Audit (human review)".

Audit toàn bộ test case AI đề xuất bằng bảng sau:

TC	Nhãn	Nhận xét hoặc chỉnh sửa
AI-01	<code>VALID</code> , <code>INVALID</code> hoặc <code>INCOMPLETE</code>	...
AI-02	...	...
...	...	...

Quy tắc:

- Gắn nhãn **VALID** / **INVALID** / **INCOMPLETE** cho **mọi** test case để đảm bảo chất lượng kiểm duyệt.
- Sửa ít nhất một test case **INVALID** hoặc **INCOMPLETE**. Nếu tất cả đều hợp lệ, chỉ ra một giả định mà AI chưa nêu rõ và bổ sung nó.
- Giải thích lý do cho mỗi nhãn (tối thiểu 1 câu).

6. Bước 3 — Extend

Thực hiện Bước 3 "Extend" với mục tiêu bổ sung ≥ 2 test case tự viết.

Tự bổ sung ≥ 2 **test case** mà AI đã bỏ sót. Với mỗi case, giải thích ngắn vì sao AI bỏ sót (prompt quality, model limitations, hoặc đặc điểm API).

Ví dụ các hướng AI thường bỏ sót:

- Response header **Content-Type** phải là **application/json**.
- Response time dưới ngưỡng chấp nhận.
- Tham số đầu vào với giá trị edge case (chuỗi rỗng, số âm, số rất lớn, ký tự đặc biệt).
- Trạng thái phản hồi không theo chuẩn REST (ví dụ trả **200** thay vì **404** khi không tìm thấy tài nguyên).

7. Bước 4 — Execute (Postman + Newman)

Thực hiện Bước 4 "Execute".

B4.1 — Khởi động provider

Mở terminal thứ nhất tại thư mục backend của dự án eshop-sut:

```
cd backend
npm run dev
```

Kiểm tra tại terminal thứ hai:

```
curl http://localhost:3000/api/products/1
```

Kết quả mong đợi: Trả về thông tin sản phẩm iPhone 15 Pro Max (status code 200).

B4.2 — Tạo iteration data

Chọn **5 test case** từ danh sách bạn đã audit + tự bổ sung ở các Bước 2 & 3 cho API đã chọn.

Tạo tệp dữ liệu chạy thử đặt tên theo dạng `mini-<api-name>.data.json` (ví dụ `mini-login.data.json`, `mini-cart.data.json`), điền các giá trị kiểm thử tương ứng. Đảm bảo cấu trúc JSON trùng khớp với các kịch bản kiểm thử và các biến mà bạn sẽ sử dụng trong Postman Collection.

B4.3 — Cấu hình header X-Student-Id và viết assertion

Tạo Collection và Environment mới trong Postman cho API đã chọn. Thêm environment variable:

```
studentId = <MSSV của bạn>
```

Trong pre-request script của request của API đã chọn trong Postman, thêm:

```
pm.request.headers.upsert({
  key: "X-Student-Id",
  value: pm.environment.get("studentId"),
});
```

Trong test script của cùng request, tự viết thêm **một assertion** kiểm tra `Content-Type` hoặc response time. Ví dụ assertion chỉ dùng để tham khảo cấu trúc:

```
pm.test("[MINI] Response is JSON", () => {
  pm.expect(pm.response.headers.get("Content-Type")).to.include(
    "application/json",
  );
});
```

Chạy Collection Runner với tệp dữ liệu đã tạo ở B4.2. Collection nên dùng assertion dựa trên biến từ data file (ví dụ `expected_status`), nên 5 iteration có thể bao gồm cả positive và negative case.

B4.4 — Chạy Newman

Export collection và environment sau khi chỉnh sửa thành:

```
mini-<api-name>.postman_collection.json
mini-local.postman_environment.json
```

Chạy:

```
newman run mini-<api-name>.postman_collection.json \
  --environment mini-local.postman_environment.json \
  --iteration-data mini-<api-name>.data.json \
  --reporters cli,json \
  --reporter-json-export mini-newman-report.json
```

Checkpoint:

- Có đúng 5 iteration cho API đã chọn.
- Không có assertion fail.
- `mini-newman-report.json` tồn tại.
- Console hoặc Postman Console cho thấy request có `X-Student-Id` đúng MSSV.

8. Bước 5 — CI/CD

Thực hiện tích hợp quy trình chạy kiểm thử vào CI/CD — tạo hai sample commits minh họa trạng thái build thành công (pass) và thất bại (fail).

Tạo file workflow `newman-api-test.yml` trong thư mục `.github/workflows/` của repository. Workflow này cần tự động khởi động Provider (eshop-sut backend), cài Newman, chạy collection với data file và upload report.

C1 — Commit pass

Commit và push bài làm lên nhánh riêng của bạn (ví dụ `feature/<MSSV>`) trên repository nhóm đã fork từ `eshop-sut`. Mở tab **Actions** trên GitHub, chọn đúng nhánh của bạn và chờ workflow `Newman API tests` chạy hoàn thành.

Chụp ảnh kết quả pipeline **pass** (tất cả test đều xanh). Lưu ảnh: `ci-pass.png`.

C2 — Commit fail (có chủ đích)

Sửa một giá trị kỳ vọng trong data file (ví dụ đổi `expected_status` từ `200` thành `999`) để gây assertion fail. Commit và push.

Chờ pipeline chạy lại. Chụp ảnh kết quả **fail** (có ít nhất một test đỏ). Lưu ảnh: `ci-fail.png`.

C3 — Khôi phục

Sửa lại giá trị đúng, commit và push lần cuối. Bài chỉ hoàn thành khi pipeline trở lại trạng thái pass.

Checkpoint:

- Có hai ảnh: `ci-pass.png` và `ci-fail.png`.
- Commit cuối cùng trên nhánh phải pass.

9. Bước 6 — Postman features

Sử dụng các tính năng hữu ích trong Postman để thiết lập bộ kịch bản kiểm thử.

Trong `test-design.md`, thêm một bảng liệt kê các Postman features bạn đã dùng trong bài:

Feature	Đã dùng?	Ghi chú
Collections	Có / Không	
Environment variables	Có / Không	
Collection variables	Có / Không	
Pre-request scripts	Có / Không	
Test scripts (assertions)	Có / Không	
Data-driven runs (Collection Runner + data file)	Có / Không	
Newman CLI	Có / Không	
Monitors	Có / Không	
Mock servers	Có / Không	
Workspaces	Có / Không	

Đánh dấu "Có" cho feature đã dùng và viết ghi chú ngắn (1 câu). Bài tập bắt buộc ít nhất 6 feature.

10. Thành phần bài nộp

Nộp một file `.zip` tên `<MSSV>_Mini_API_Testing.zip` gồm các thành phần sau:

- `test-design.md`: prompt, AI output rút gọn, bảng audit, test case tự bổ sung (extend), và bảng Postman features.
- `mini-<api-name>.data.json` — tệp dữ liệu kiểm thử.
- `mini-<api-name>.postman_collection.json` và `mini-local.postman_environment.json`.
- `mini-newman-report.json`.
- `newman-api-test.yml` — file workflow CI/CD.
- Hai ảnh: `ci-pass.png` và `ci-fail.png`.

11. Tài liệu tham khảo

- Newman command-line options: <https://github.com/postmanlabs/newman#command-line-options>.
- GitHub Actions documentation: <https://docs.github.com/en/actions>.
- Postman Sandbox API Reference (cho việc viết test script & assertions): <https://learning.postman.com/docs/writing-scripts/script-references/postman-sandbox-api-reference/>
- Postman Data-driven Testing Guide (sử dụng iteration data files): <https://learning.postman.com/docs/sending-requests/runner/running-multiple-iterations/>
- Newman Command-line Integration Guide: <https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/>
- GitHub Actions Setup-Node Action (sử dụng trong CI workflow): <https://github.com/actions/setup-node>

